Microsoft

AutoChecklist

Automated Checklist Refinement for LLM Judges

Checklists that improve themselves — turning a free-form comment box into evaluation criteria that adapt to your data.

Mansi Uniyal · Mukul Singh · Gust Verbruggen · Vu Le · Sumit Gulwani

THE PROBLEM

Evaluating LLM output at scale is hard

— **LLMs are everywhere in production**
summarization, translation, Q&A, content generation.

— **Classic metrics fall short.**
BLEU and ROUGE don't capture semantic quality.

— **Human judgment is the gold standard**
but it's costly, slow, and inconsistent.

— **LLM-as-a-judge scales**, yet
struggles with reproducibility, transparency, and bias.

THE CORE TENSION

We need evaluations that are…

Scalable like an automated metric

Interpretable like a human rubric

Reliable across domains & data

Pick all three — that's the goal.

A PROMISING ANSWER

Checklists make judging objective

Microsoft

Decompose a fuzzy judgment into specific, verifiable yes/no questions.

Checklist for a translation

- ✓ Is the query clear about the desired outcome?
- ✓ Is it free from unstated assumptions?
- ✓ Is it free from conflicting instructions?
- ✓ Is the meaning accurately conveyed?

— **Objective** — each item checks one concrete property.

— **Interpretable** — you can see why a score was given.

— **Scalable** — an LLM can answer the checklist for thousands of items.

— **Auditable** — humans and models share the same rubric.

BUT THERE'S A CATCH

A good checklist is hard to write

— **Checklists are manually curated**
crafting effective ones takes expert iteration.

— **They're static.**
You can never be sure they cover every failure case.

— **They miss dataset-specific quirks**
the designer never anticipated.

FAILURE A STATIC CHECKLIST MISSES

Excel task: "Apply conditional formatting from cell C3:C15 of Sheet1."

The instruction names a **range** but no **format** — it's ambiguous, yet no checklist question separates the two mistakes.

Completeness can only be judged from the data — not from a fixed, hand-written list.

THE KEY IDEA

Take free-form comment box from human surveys.

Surveys always leave room for “anything else?”. We give the LLM judge the same option — then turn those free-form comments into **new checklist questions**, so the checklist evolves with the data it actually sees.

① COMMENT

Judge flags anything odd, wrong, or notable.

② AGGREGATE

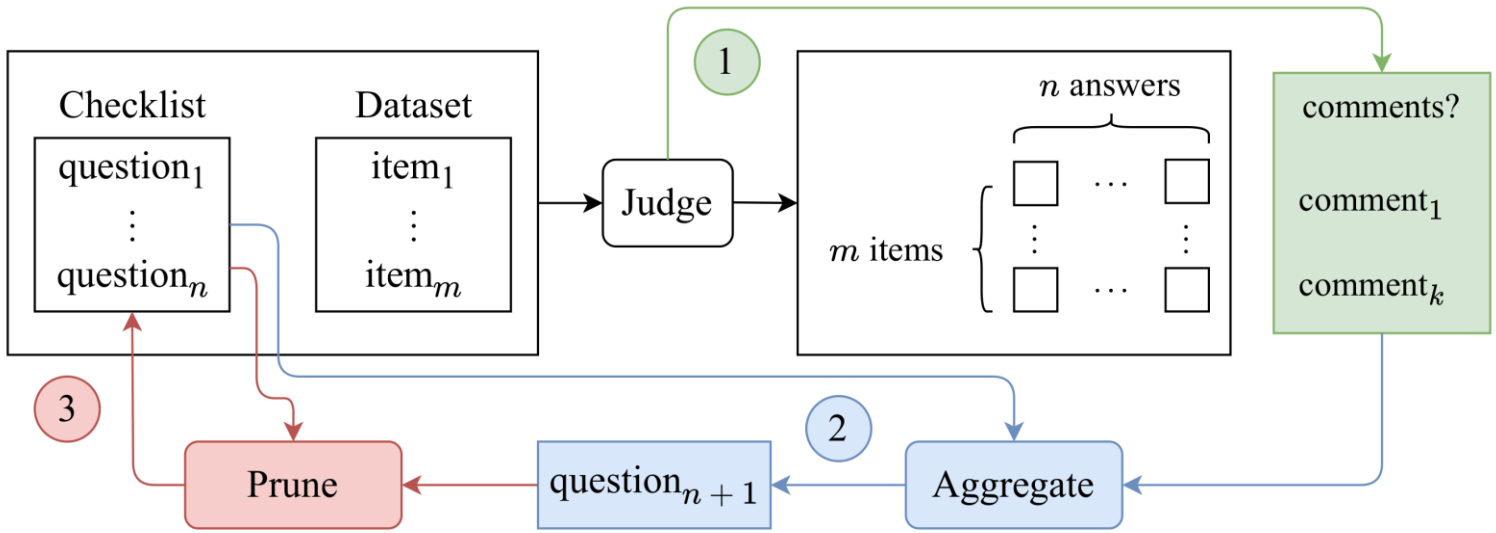
Comments become the new checklist questions.

③ PRUNE

Redundant questions are removed and summarized for optimization.

THE SYSTEM

AutoChecklist: a self-refining loop



1 Generate

Judge answers the current checklist plus a free-form comment.

2 Aggregate

Comments are merged into new checklist questions.

3 Prune

Semantically redundant questions are dropped, and summarized.

Repeat until no new comments — or a max iteration hits.

UNDER THE HOOD

Three small components, one prompt change

Microsoft

① Free-form comments

- Add one line to the judge prompt: “comment if you spot anything interesting.”
- Meta-rules: flag wrong/correct patterns or discrepancies; don't explain the answer.

② Aggregate into questions

- An LLM turns comments into criteria: avoid duplicates, verify one property each,
- split compound questions (“clear & concise” → two questions), stay objective.

③ Prune

- Feed existing + new questions to the model; remove the semantically equivalent ones.
- Iterative pruning keeps the list tight (~10 questions) with controlled convergence.

HOW WE EVALUATE IT

Applicability across domains

Microsoft

Translations

WMT22 German→English news, with human direct-assessment scores (0–100).

Instructions

InstructExcel — Excel task descriptions scored 0–3 on ambiguity.

Summaries

SummEval news summaries rated on coherence, consistency, fluency, relevance.

Model gpt-4o, temperature 0

Sample 100 tasks per dataset, 5-fold CV, <6 iterations

Metric Δ Pearson correlation with human ratings — do new questions capture meaningful failure modes?

RESULT · RQ1

The comment improves judgments

Microsoft

FSE 2026

> 90%

partial match with no-comment run

answers

only change when the comment makes them better

No cost

comments never degrade the original checklist

WHY THE COMMENT HELPS — A REAL CASE

Query: "Heighlight value where it is greater than 3 on G9:G17 on Sheet1."

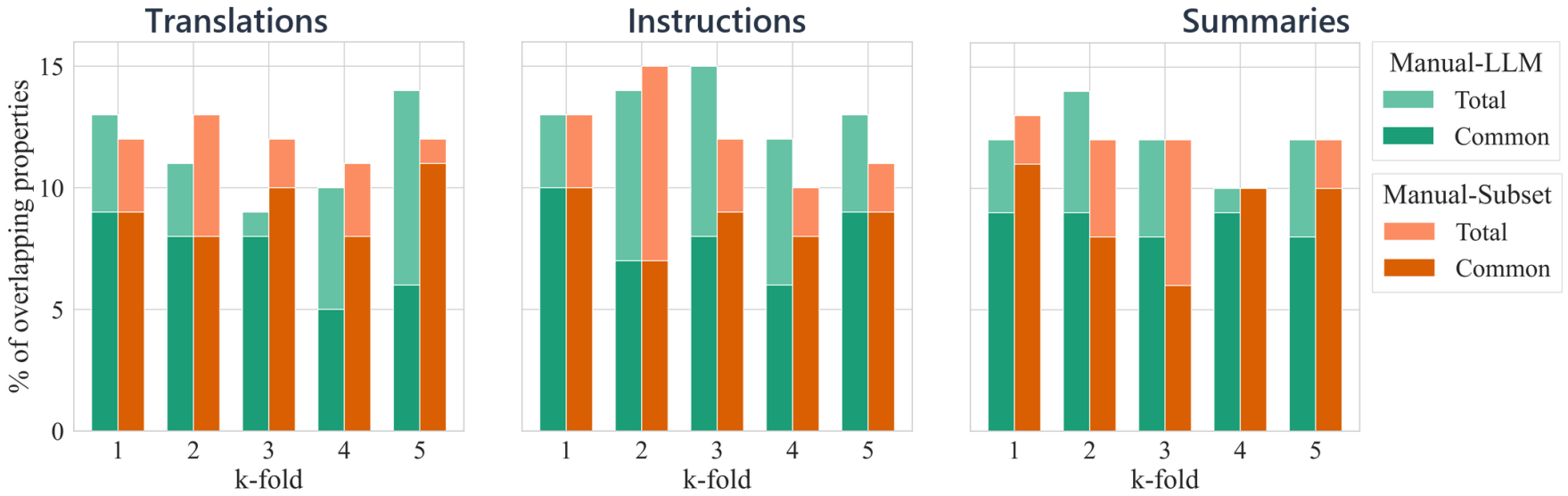
Without comments → judge answers **"no, it's ambiguous."**

With comments → **"no — but only because 'heighlight' is a typo, not real ambiguity."** The comment exposes the true reason, so the model stops over-clustering unrelated failures.

RESULT · RQ2

Any seed converges to the same checklist

Start from a one-line generic seed, a synthetic list, or a large hand-built one — the loop lands in the same place.



— **Heavy overlap.** Across 5 folds, refined checklists overlap strongly with both manual references.

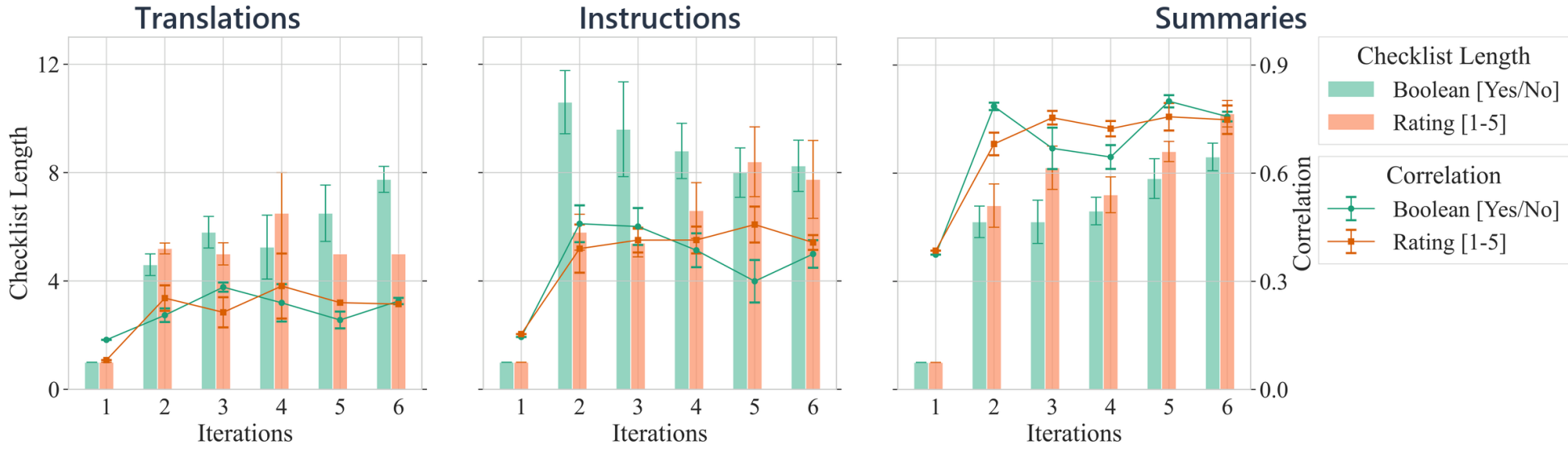
— **One line is enough.** A single-question seed already recovers most properties a large hand-built seed finds.

— **Seed-agnostic.** Added questions come from the data, not from how the checklist was initialized.

RESULT · RQ2

Binary or 1–5? Either format works

We reran the loop with Yes/No questions and with 1–5 rating scorecards.



— **Comparable accuracy.** Both formats reach similar correlation with human ratings in every domain.

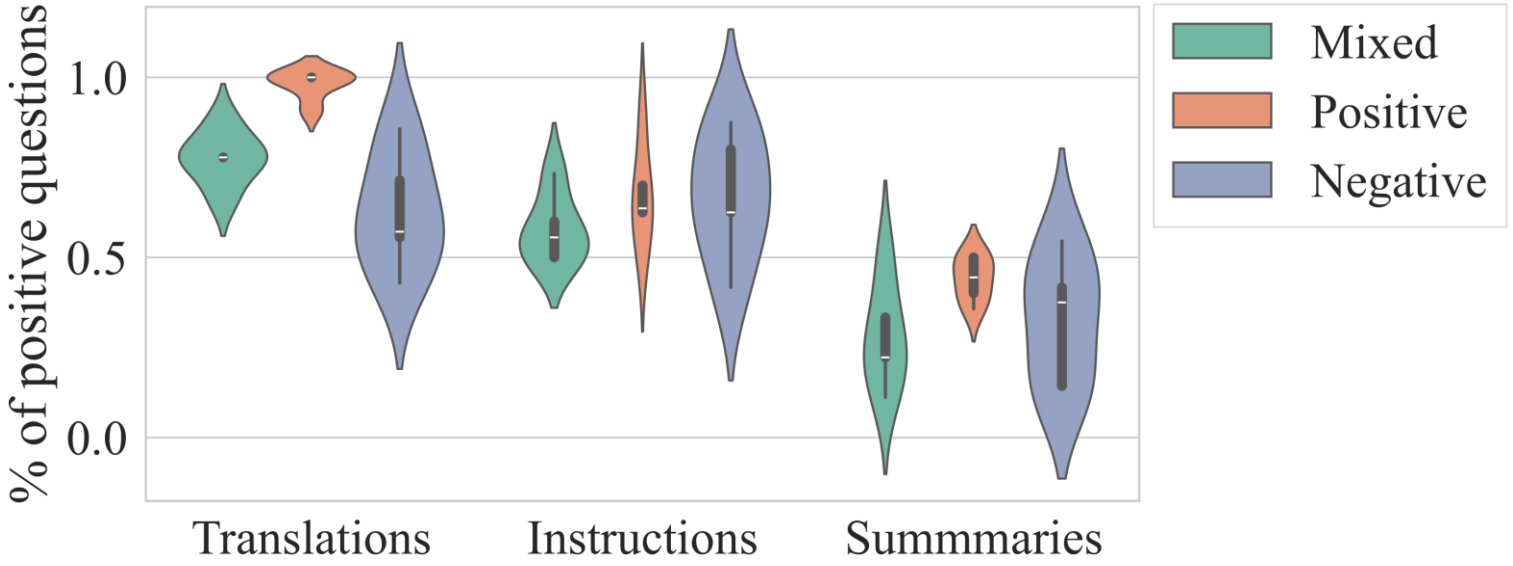
— **Similar growth.** Checklist length evolves the same way regardless of the response format.

— **Free choice.** Teams keep whichever format their evaluation pipeline already uses.

RESULT · RQ2

The loop self-corrects its phrasing

LLMs are sensitive to wording — "is it concise?" vs. "does it avoid being lengthy?". We seeded positive, negative, and mixed tones.



— **Same destination.** Whatever the seed tone, the final checklist settles into a balanced, mixed tone.

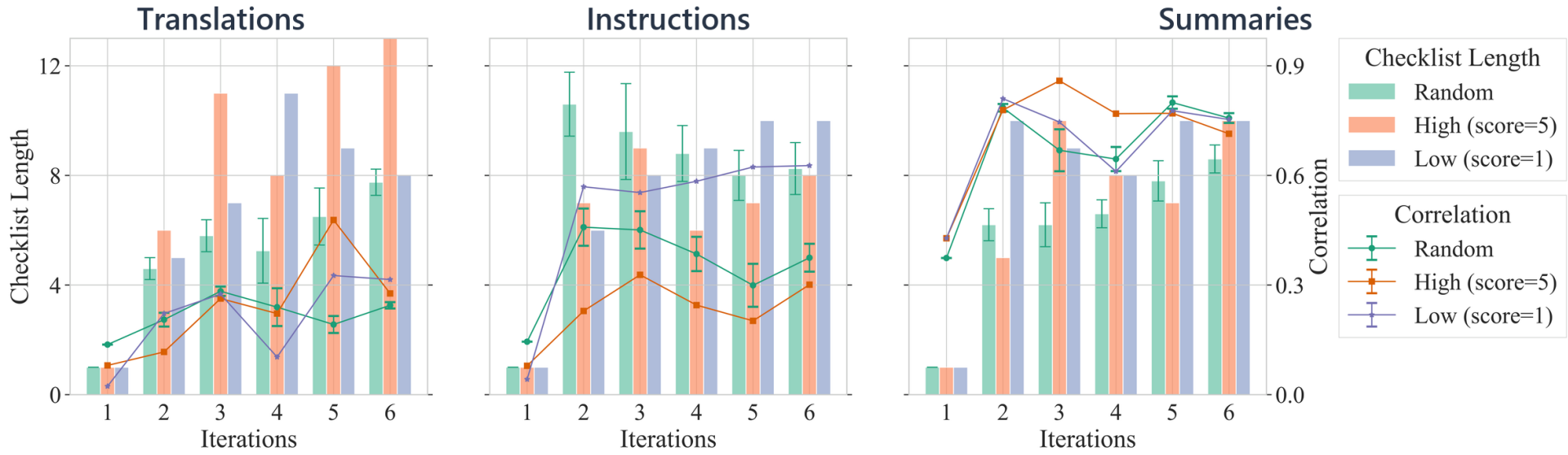
— **Bias correction.** Positive-only and negative-only seeds drift back toward an even split of phrasings.

— **Robust by design.** Phrasing bias in the seed does not propagate to the refined checklist.

RESULT · RQ3

What you refine on matters

We refined on a random sample, on only top-scoring examples, and on only worst-scoring examples.



— **Random wins.** Refining on a random sample converges best and stays the most stable.

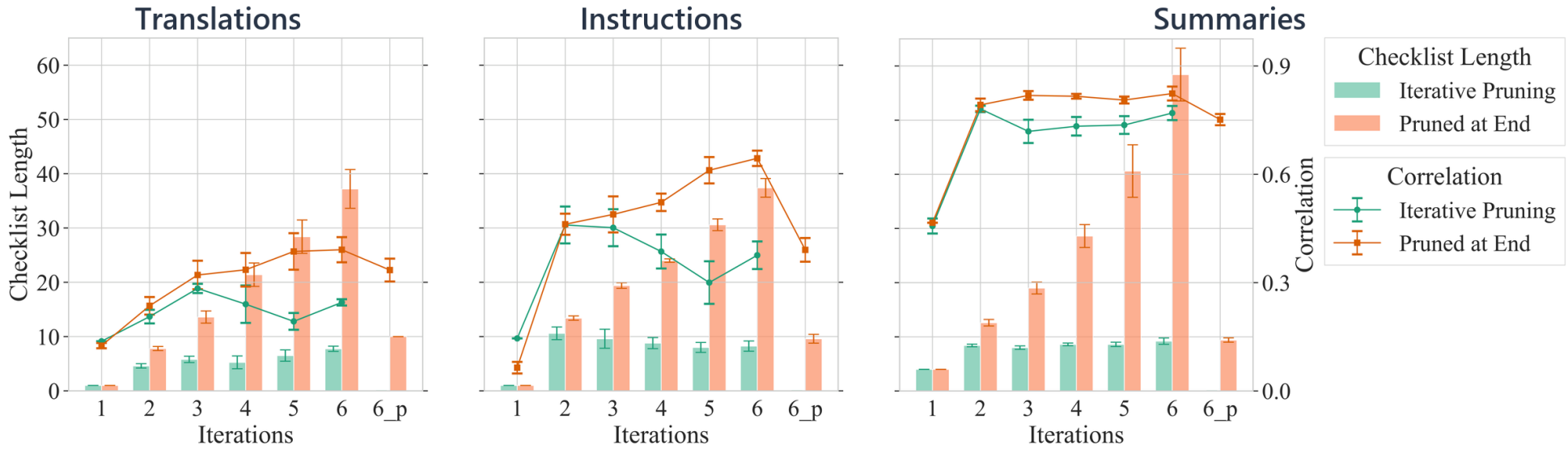
— **Skew hurts.** Best-only or worst-only slices yield overly specific questions and unstable metrics.

— **Represent the data.** The loop needs the full quality range to generalize.

RESULT · RQ3

Pruning keeps the checklist tight

Prune a little on every iteration, or let the checklist grow and prune once at the end.



— **Stay lean.** Iterative pruning holds the checklist near ten questions with smooth convergence.

— **Coverage vs. quality.** Pruning only at the end gives broader coverage but slightly lower correlation.

— **Stable criteria.** Both strategies share 7 of 8 core properties — same aspects, different wording.

WHY IT MATTERS

From hand-crafted rubrics to data-driven evaluation

Subjective → sub-objective

Converts fuzzy quality judgments into concrete, verifiable questions.

Robust & reproducible

Stable across seeds, tones, response formats, and datasets variations

Less manual effort

Automates the slow, expert-heavy work of writing and maintaining checklists.

Deployable

Runs internally to build reliable evaluation pipelines — not just a paper prototype..

Thank you.

AutoChecklist — checklists that improve themselves.

Mansi Uniyal · mansiuniyal@microsoft.com

Microsoft · FSE 2026 Industry, Montréal

Open to Questions